



TXW80X SDK Quick Start Manual



Notice

Due to product version upgrades or other reasons, this document will be updated from time to time. Unless otherwise agreed, this document only

As a guide for use without any warranty.

Zhuhai Taixin
Semiconductor Co.,Ltd

3rd Floor, Building 11, Science and Technology Innovation Park, No. 1, Gangwan, High-tech Zone, Zhuhai City

Confidentiality level	A	TXW80X SDK Quick Start Manual	File No	TX-0000
Release date	2022-07-26		file version	V1.1

Revision history

date	Version	describe	Revised by
2022-07-26	V1.1	1. Add CDK download code and mass production burning description 2. Add XIP solution code efficiency debugging and optimization description	TX
2022-05-25	V1.0	initial version	TX



Zhuhai Taixin Semiconductor Co., Ltd.
Zhuhai Taixin Semiconductor Co.,Ltd

3rd Floor, Building 11, Science and Technology Innovation Park, No. 1, Gangwan, High-tech Zone, Zhuhai City

Copyright infringement will be investigated

Copyright © 2022 by Tai Xin All rights reserved

Confidentiality level	A	TXW80X SDK Quick Start Manual	File No	TX-0000
Release date	2022-07-26		file version	V1.1
Table of Contents TXW80X SDK Quick Start Manual..... 1 1. Overview 1 2. Integrated development environment.. 1 2.1. CKLink Debugger..... 1 2.2. CDK Development Environment..... 2 2.3. Compilation and debugging..... 2 2.3.1. Compilation. 2 2.3.2. Debugging..... 2 2.3.3. Inserting debugging in the middle.....3 2.4. Firmware burning. 5 2.4.1. CDK download code..... 5 2.4.2. CKLink Programming..... 5 2.4.3. UART programming..... 6 2.4.4. Mass production programming..... . 6 3. Hardware development board..... . 7 3.1. Audio and video development board..... 7 3.1.1. Audio and video development board interface introduction... ..7 3.2. Wireless network card development board..... 8 3.3. Low-power development board..... 8 3.4. IOT development board 8 3.5. Bridge development board..... 8 3.6. Touch development board.. 8 4. SDK Overview..... 9 4.1. SDK Feature..... 9 4.2. SDK framework diagram..... 9 4.3. Directory structure..... 10 4.4. SDK startup process..... 11 4.5. System parameters 12 4.6 .FLASH LAYOUT..... 14 4.7. Memory layout..... 15 4.8. Basic rules for solution development16 4.9. Driver API usage instructions18 4.10. OSAL API usage instructions.....18 4.10.1. Task..... 18 4.10.2. Mutex..... 21 4.10.3. Semaphore.....21 4.10.4. Msgqueue..... 22				
		Zhuhai Taixin Semiconductor		3rd Floor, Building 11, Science and Technology Innovation Park, No. 1, Gangwan, High-tech Zone, Zhuhai City
		Co.,Ltd		

Confidentiality level	A	TXW80X SDK Quick Start Manual	File No	TX-0000
Release date 2022-05-25			file version	V1.0
4.10.5. SoftTimer.....24 4.11. Debugging information..... 25 4.11.1. WiFi protocol stack debugging information output..... .25 4.11.2. System Heap usage.....26 4.11.3. CPU usage.....26 4.11.4. XIP instruction fetch efficiency.....27 5. Other learning resources.....28				
		Zhuhai Taixin Semiconductor Co., Ltd. Zhuhai Taixin Semiconductor Co.,Ltd		3rd Floor, Building 11, Science and Technology Innovation Park, No. 1, Gangwan, High-tech Zone, Zhuhai City
Copyright infringement will be investigated				
Copyright © 2022 by Tai Xin All rights reserved				

1 Overview

This article is written for program software design developers to help you quickly get started with the SDK.

This document is mainly applicable to the following

engineers: ỹ Technical support

engineers ỹ Solution software development engineers

Product scope applicable to this document:

model	encapsulation	Package
TXW806		
TXW803		
TXW802		
TXW801	QFN24	

2. Integrated development environment

2.1. CKLink Debugger

CKLink Lite is the debugging tool for the SDK of the TXW80X chip/module and can be purchased at Pingtou Ge's Taobao official store.



Figure 2.1.1 - Debugger

2.2. CDK development environment

CDK is the integrated development environment of Pingtoug CPU, which can be downloaded and installed from the Pingtoug official website. This SDK Developed based on CDK v2.8.8, it is recommended to use this version or higher for development.

If you cannot download it, please contact our FAE.

2.3. Compilation and Debugging

2.3.1. Compilation

After the CDK is installed, double-click the SDK project path to open the project *.cdkproj, and click Edit in the CDK menu.

Translate or shortcut key F7 to execute compilation.

2.3.2. Debugging

When debugging, it is recommended to set ICE Clock 120Khz, check reset after connect, and use softreset. Please turn off the watchdog in the code when debugging with breakpoints. The specific operation steps are as follows (shown in Figure 2.3.2.1):

1. Enter project setting/debug and select Use ICE;
2. Enter the setting. If the debug board can be connected correctly, enter the Connected Debug Target box.
You can read Target chip Info;
3. Set the ICE Clock to 120Khz (if you need the debugger to set the frequency higher, please refer to the FAQ), then
Click OK to save.

After setting up, connect the debugging cable of CKLink to the debugging port of the board, click the debugging button of CDK or CTRL+F5, CDK will start debugging.

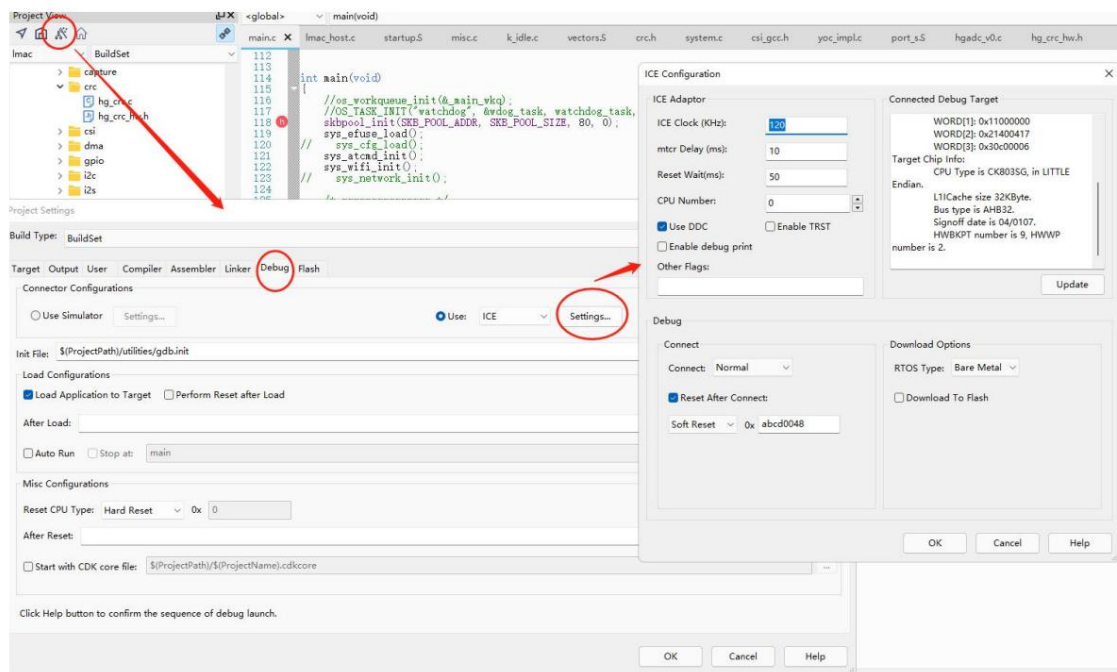
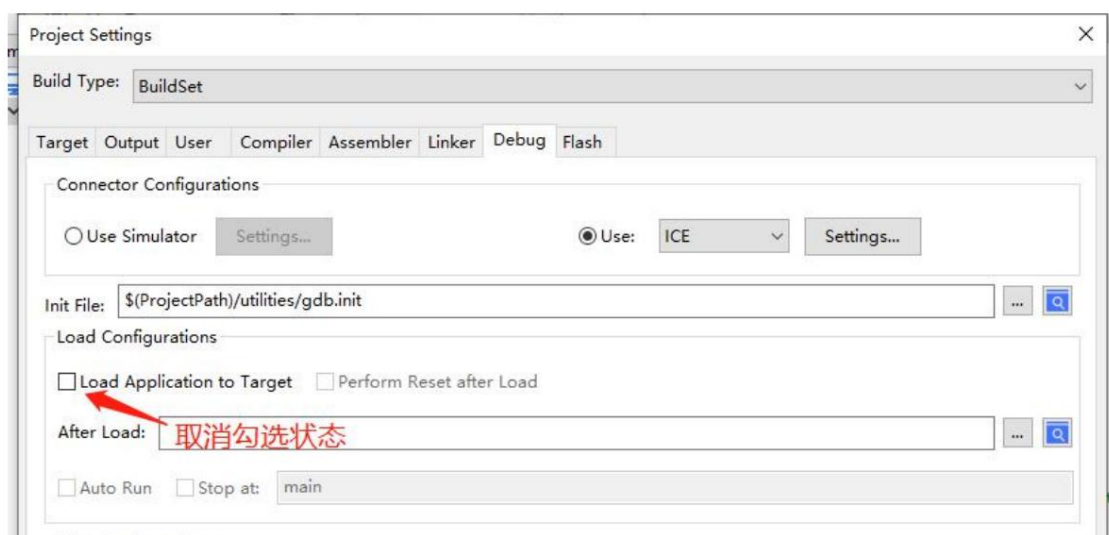


Figure 2.3.2.1 - Debug setup

2.3.3. Inserting debugging midway

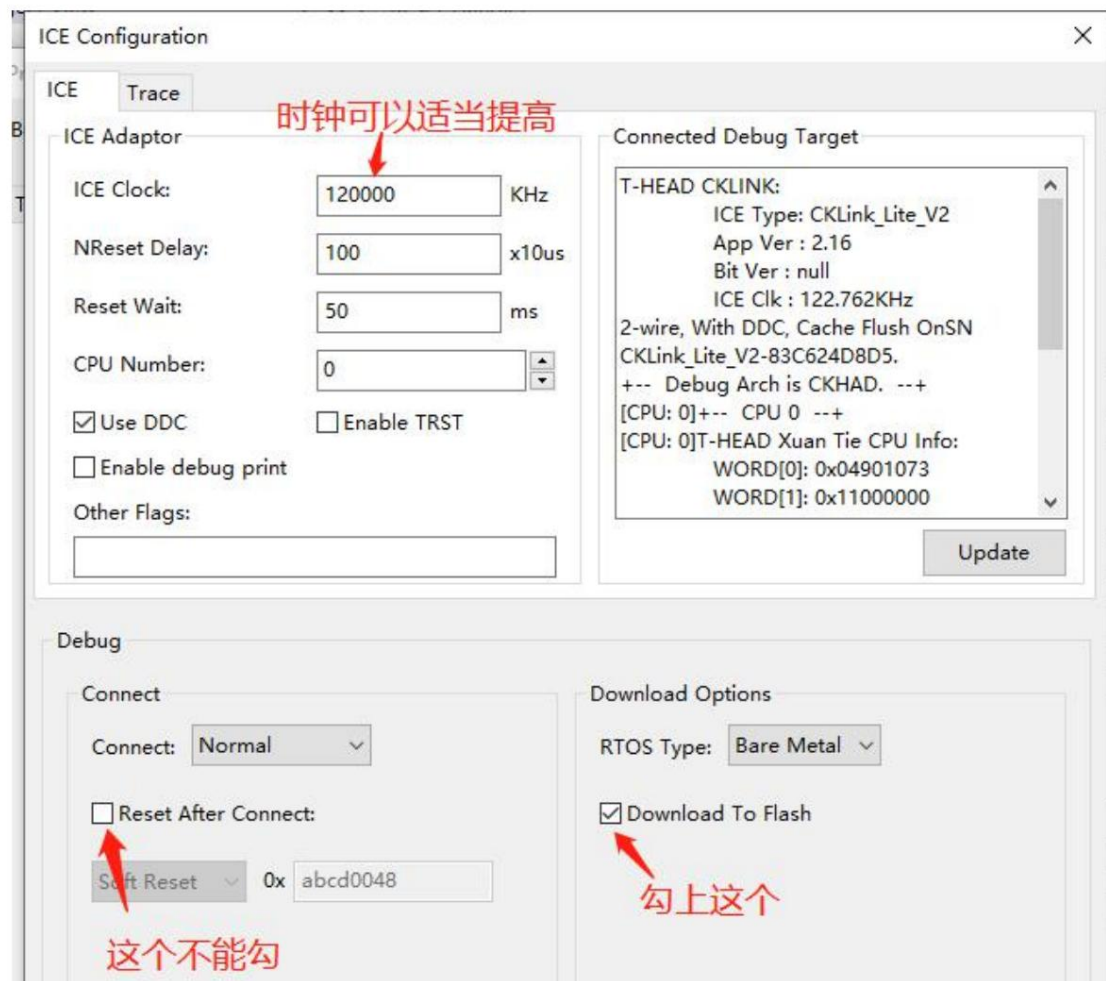
Sometimes it is necessary to insert a debugger midway for debugging. At this time, you cannot operate according to the ordinary debugging method (it will reset and damage the scene). You need to configure the SDK as follows, and then insert the debugger for debugging:

1. In the Project Settings window (can be opened using the ALT+F7 shortcut key), cancel the "Load Application to Target" check box.

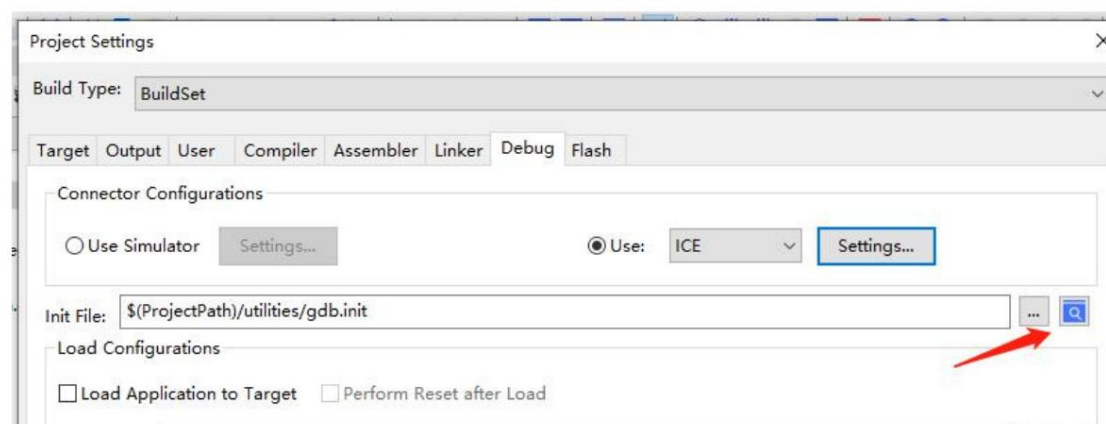


2. Continue in the Project Settings window, click Settings on the left side of ICE, and in the pop-up ICE

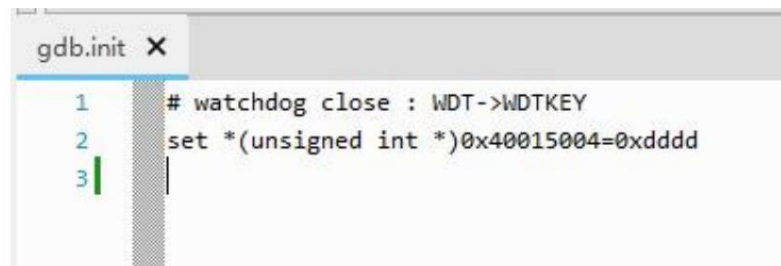
In the Configuration window, check "Download to Flash" and uncheck "Reset After Connect". In addition, you can set the value of ICE Clock according to the frequency of the system clock (as long as it is not higher than 1/2 the system clock). Finally press OK to exit the window.



3. Return to the Project Settings window, click the magnifying glass icon on the far right of Init File, and modify gdb.init content. At this point you can press OK to exit the Project Settings window.



4. Modify gdb.init according to the actual situation. If the watchdog is not enabled for the program to be debugged, you can delete all the contents in gdb.init; if the watchdog is enabled, the following two statements need to be left in gdb.init.



```

1  # watchdog close : WDT->WDTKKEY
2  set *(unsigned int *)0x40015004=0xdddd
3

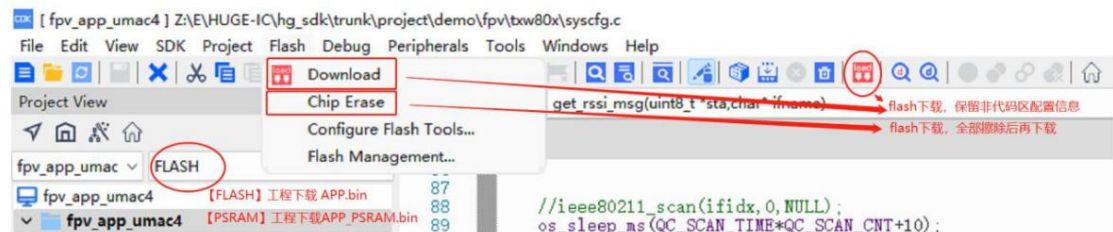
```

After the above steps are completed, you can insert the debugger and  click to enter debugging mode.

2.4. Firmware burning

2.4.1. CDK download code

After the CDK code is successfully compiled, it can be downloaded through the menu "Flash" -> "Download". Downloading in this way will keep the configuration information in the non-code area; if you need to clear the configuration information, you can choose the "Chip Erase" method to proceed load.



2.4.2. CKLink programming

The burning tool is: CSKY-FlashProgrammer-windows, download and install from Pingtou Ge's official website. It is recommended to use V1.0.14 and above. The specific programming steps are:

1. Execute CSKYFlashProgrammer.exe
2. Set Program Algorithm File and Script in advance TargetConfig
3. Check whether to run the code immediately after downloading as needed
4. Set ICE CLK in advance LocalJTAG: 12Mhz

5. Return to the main interface to add the download file and click the start button to download.

Advance

Target Config Local JTAG Read

Program Algorithm File:

Program Pre-Script File:

Erase Type: Start: 0x Length: 0x

Options

Connect: ☒ Reset and Run 0x

Command Timeout(s):

CPU Select (0,1,2,...):

☐ Enable Log

FLASH PROGRAMMER

File Advance

User Config: default

Target Objects

Bin

☒ Program ☒ Verify Address: 0x

CRC16: 0x525B CRC32: 0x2E52869E (733,712)

Bin

☐ Program ☐ Verify Address: 0x

CRC16: 0xAF0E CRC32: 0x91D2EE29 (25,104)

version: 1.0.14

2.4.3. UART programming

The SDK integrates the serial port upgrade function. If it has been turned on, you can upgrade through the serial port command "TX+OTA". Please consult our FAE for specific methods.

2.4.4. Mass production burning

For details on mass production and programming, please refer to "TXW80X Mass Production and Programming Guide.pdf"

3. Hardware development board

For quick start and solution evaluation, we provide development boards for various application scenarios.

3.1. Audio and video development board

3.1.1. Audio and video development board interface introduction

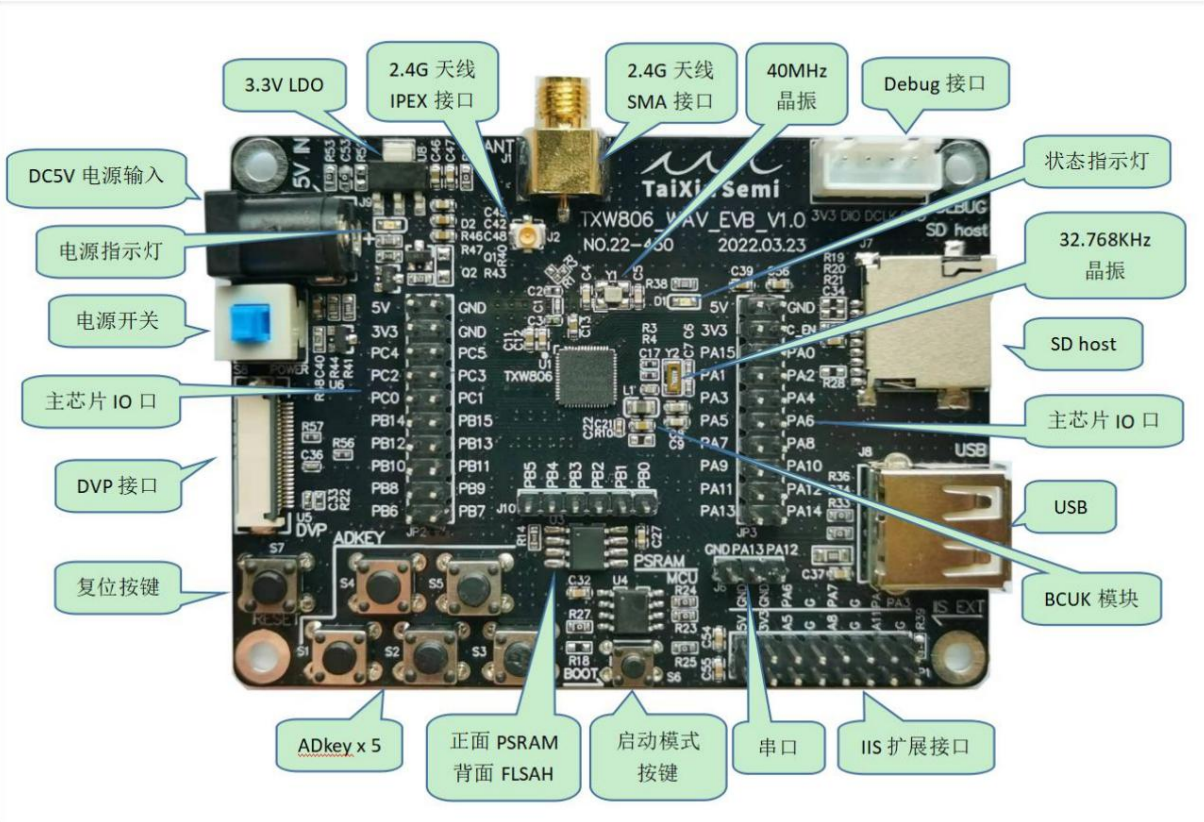


Figure 3.1.1.1 - Main view of audio and video development board

Special Instructions

Mode start button: This button can save the system with one click, run to death when the chip is powered on, cliknk programming and other upgrades

Used when all levels are disabled. Note that the Debug interface cannot be connected when pressing this button.

IIS extension interface: This interface is used to expand the audio daughter board.

3.2. Wireless network card development board

3.3. Low power consumption development board

3.4. IOT development board

3.5. Bridge development board

3.6. Touch development board

4. SDK Overview

4.1. SDK Feature

• Application scenarios

• VGA/720P otoscope, aerial photography

• miniDV

• Video doorbell

• Wi-Fi touch home appliances

• Operating system

• AliOS

• Job role

• AP

• STA

• Network protocol stack

• LWIP

• MQTT

• HTTP

• File system

• (ex)FAT

• Abstract data bus

• USB device•SDIO device•SPI salve•UART•I2C

• Abstract peripherals

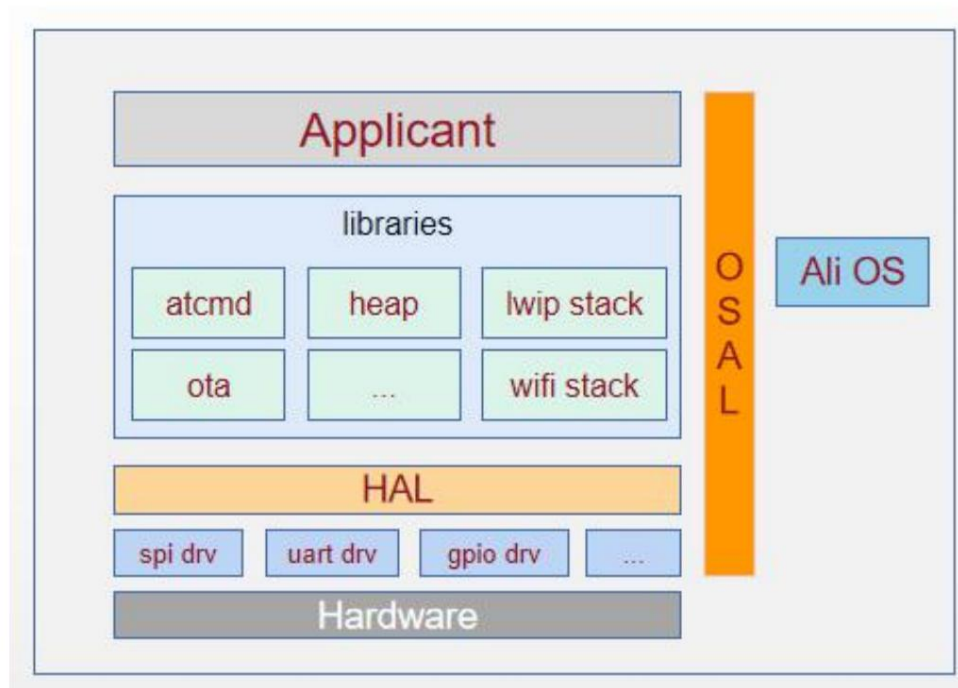
• GPIO•ADC•DMA•DVP•MJPEG•IIS•PDM•USB•SDIO •SPI•UART•I2C•LED•

PWM•UART•WDT•TIMER•TOUCH•AES•CRC

4.2. SDK framework diagram

The basic framework of the SDK is shown in the figure below. The SDK defines two abstract layers: OSAL and HAL; the OSAL abstract layer defines API for OS related operations, HAL defines hardware device access API.

When using the SDK for secondary development code, when it comes to API calls related to OSAL and HAL, please use OSAL. and API provided by HAL



4.3. Directory structure

The directory structure after decompression of TXW80x SDK is shown in the figure below.



• `csky`: CSKY RTOS code directory, usually there is no need to modify this part of the code. • `project`:

CDK project application code directory, `main.c` is located in this directory. • `sdk/chip`: chip definition

and startup code directory • `sdk/driver`: peripheral driver code

directory • `include`: SDK header file directory • `lib`: commonly

used library, including lwip protocol stack, Ethernet

PHY driver, etc. Code libraries transplanted by users can be placed in this directory.

4.4. SDK startup process

The startup code of the SDK is in the `sdk/chip/txw80x` directory. The general startup process is as shown in the figure below, where

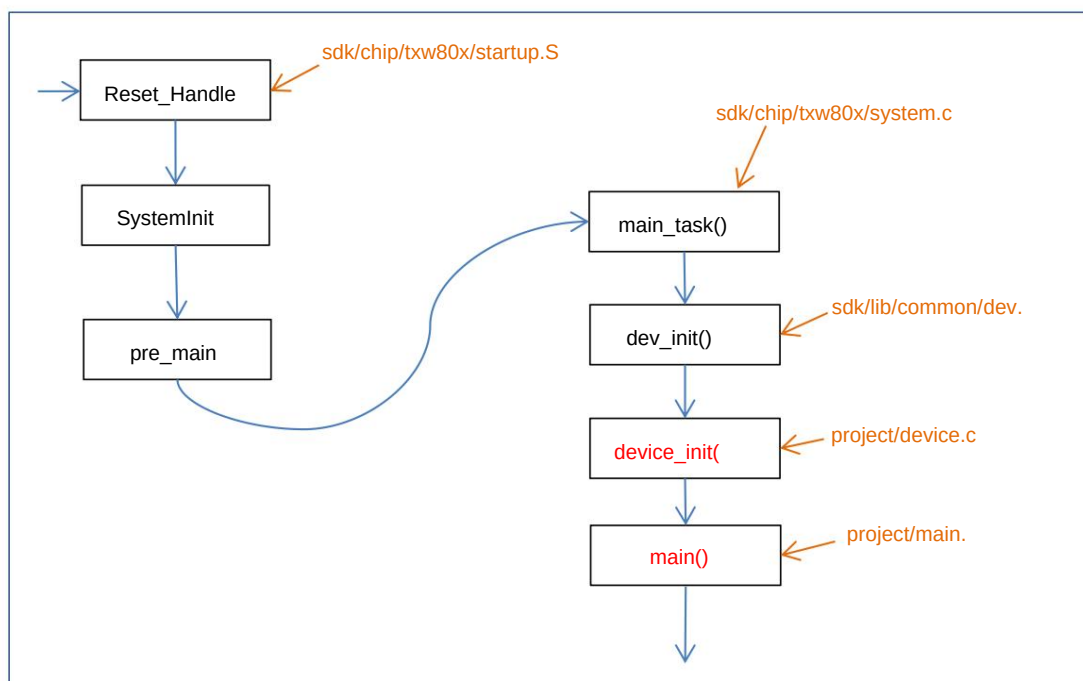
The `device_init` and `main` functions can be modified accordingly, but other startup process codes should not be modified.

• `device_init` function: System device initialization function, which completes driver initialization and device registration; all devices only

It can be used only after registration. In this function, you can delete the initialization code of unused devices to reduce the code body.

product.

• `main` function: application entry function.



4.5. System parameters

The system parameters of the SDK are defined in the project/sys_cfg.h file, as shown below:

```

11: struct sys_config {
12:     ... uint16 magic_num, crc;
13:     ... uint16 size, rev1, rev2, rev3;
14:     ... //////////////////////////////////////
15:
16:     ... uint8 wifi_mode, bss_bw, tx_mcs, channel;
17:     ... uint8 bssid[6], mac[6];
18:     ... uint8 ssid[32];
19:     ... uint8 key[32];
20:     ... uint16 bss_max_idle, beacon_int;
21:     ... uint16 ack_tmo, dtim_period;
22:     ... uint32 key_mgmt;
23:
24:     ... uint32 dhcpc_en:1, dhcpd_en:1, xxxxxx:30;
25:     ... uint32 ipaddr, netmask, gw_ip;
26:     ... uint32 dhcpd_startip, dhcpd_endip, dhcpd_lease_time;
27:     ...
28: };
29:

```

• The first 12 bytes: magic_num, crc, and size are core fields and cannot be modified, and application code cannot use these fields; rev1, rev2, and rev3 are SDK reserved extension fields and are not recommended. The system parameters are configured in project/sys_cfg.c as follows:

```

24:
25: struct sys_config sys_cfgs = {
26:     ... wifi_mode = WIFI_MODE_AP,
27:     ... channel = CHANNEL_DEFAULT,
28:     ... beacon_int = 100,
29:     ... dtim_period = 1,
30:     ... bss_max_idle = 300,
31:     ... key_mgmt = WPA_KEY_MGMT_PSK,
32:     ... ipaddr = 0x0101A8C0, //192.168.1.1
33:     ... netmask = 0x00FFFFFF, //255.255.255.0
34:     ... gw_ip = 0x0101A8C0, //192.168.1.1
35:     ... dhcpd_startip = 0x0201A8C0, //192.168.1.2
36:     ... dhcpd_endip = 0xFE01A8C0, //192.168.1.254
37:     ... dhcpd_lease_time = 3600,
38:     ... dhcpd_en = 1,
39:     ... dhcpc_en = 1,
40: };
41:

```

• wifi_mode: The field is the Wi-Fi working mode, supporting STA, AP, AP+STA.

• Channel: 2.4G channel selection, 0 means automatic selection, non-0 means forced channel (STA cannot force channel)

• beacon_int: beacon interval (ms). • dtim_period: dtim interval (number of beacons). • bss_max_idle: The maximum inactivity time within the bss range. The AP will query whether the STA is still there after timeout.

• key_mgmt: Encryption mode selection •

Ipaddr: IP address • Netmask:

Network mask • gw_ip: Gateway •

dhcpcd_startip: The

beginning of the IP address range allocated by the DHCP server • dhcpcd_endip: The end

of the IP address range allocated by the DHCP server • dhcpcd_lease_time: DHCP IP

lease time dhcpcd_en: DHCP server enable • dhcpc_en: DHCP

client enable

• Parameter area storage location: The parameter area storage location is specified in the `syscfg_info_get` function in `device.c`. This

function specifies the flash device where each parameter area is located, the address and size of the parameter area. The SDK uses

the last two sectors of Flash by default (the sectors occupied by the two parameter areas cannot overlap).

```
int32 syscfg_info_get(struct syscfg_info *pinfo)
{
#ifdef SYSCFG_ENABLE
    pinfo->flash1 = &flash0;
    pinfo->flash2 = &flash0;
    pinfo->size = pinfo->flash1->sector_size;
    pinfo->addr1 = pinfo->flash1->size - (2 * pinfo->size);
    pinfo->addr2 = pinfo->flash1->size - pinfo->size;
    ASSERT((pinfo->addr1 & ~(pinfo->flash1->sector_size - 1)) == pinfo->addr1);
    ASSERT((pinfo->addr2 & ~(pinfo->flash2->sector_size - 1)) == pinfo->addr2);
    ASSERT((pinfo->size >= sizeof(struct sys_config)) &&
            (pinfo->size == (pinfo->size & ~(pinfo->flash1->sector_size - 1))));
    return 0;
#else
    return -1;
#endif
}
```

• Parameter API:

The parameter reading/storage API is defined in the `sdk/include/lib/syscfg.h` file. The SDK defines global variables

The variable `sys_cfgs` is used as a reference to the entire SDK parameters and can be directly accessed in other modules.

```
#ifndef _HGIC_SYSCFG_H_
#define _HGIC_SYSCFG_H_

#define syscfg_dbg(fmt, ...) //os_printf("%s:%d::"fmt, __FUNCTION__, __LINE__, ##__VA_ARGS__)
#define syscfg_err(fmt, ...) os_printf("%s:%d::"fmt, __FUNCTION__, __LINE__, ##__VA_ARGS__)

#define SYSCFG_MAGIC (0x1234)

struct syscfg_info {
    struct spi_nor_flash *flash1, *flash2;
    uint32 addr1, addr2;
    uint32 size;
};

int32 syscfg_init(struct sys_config *cfg);
int32 syscfg_read(struct sys_config *cfg);
int32 syscfg_write(struct sys_config *cfg);
int32 syscfg_info_get(struct syscfg_info *pinfo);
void syscfg_loaddef(void);

#endif
```

Among

them: • `syscfg_init`: syscfg module initialization and loading flash parameters. This function is usually in the main function

Execute at startup and read flash parameters.

ÿ syscfg_read: Read parameters. The valid parameter area will be selected from the 2 parameter areas for reading. ÿ

syscfg_write: Save parameters and use 2 parameter areas alternately. ÿ syscfg_info_get:

Specifies the storage location information of the parameter area. According to the actual flash in device.c

information to specify.

ÿ syscfg_loaddef: parameter area load default will set both parameter areas as invalid areas.

The default configuration of the system clock SDK is DEFAULT_SYS_CLK = 180Mhz. This is required if the solution requires modifying the system clock.

Rewrite function: system_clock_init, system_clock_init source code is as follows:

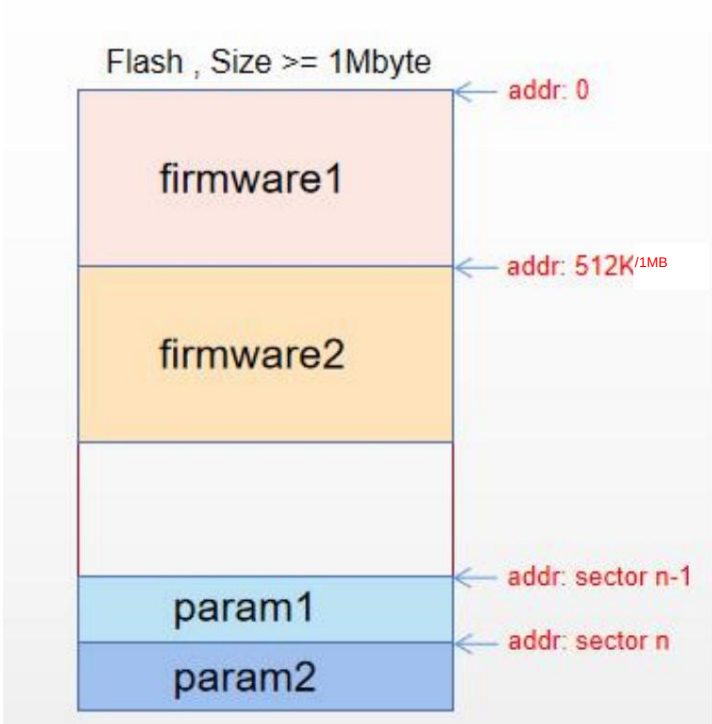
```
#define DEFAULT_SYS_CLK          180000000UL

__weak void system_clock_init(void)
{
    if (!sysctrl_cmu_sysclk_set(DEFAULT_SYS_CLK, 0)) {
        while (1);
    }
}
```

4.6. FLASH layout

The SDK is designed with dual firmware areas and dual parameter areas. The addresses of the two firmware areas are fixed at 0 and 512KB/1MB. By default, the parameter area uses the last two sectors of flash, with a size of 1 sector.

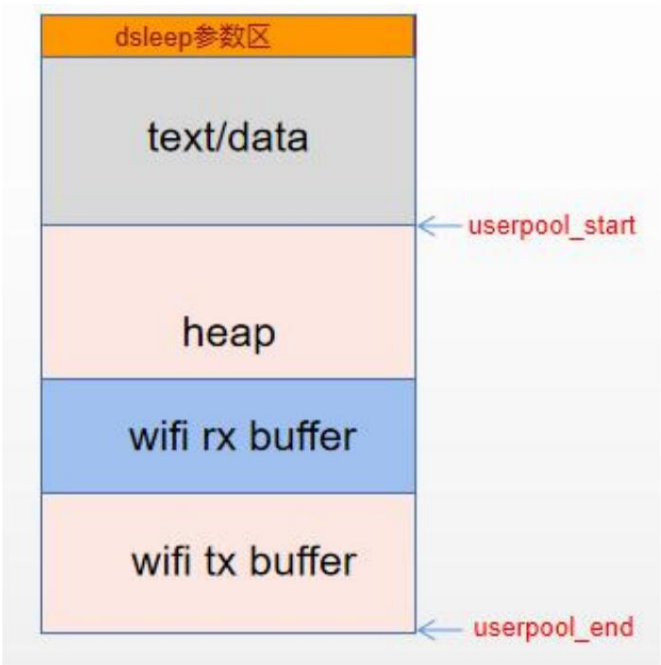
The default Flash partition is as follows:



The parameter area can be adjusted according to the actual situation.

4.7. Memory layout

The SRAM Size of the TXW80X chip is 288KByte, and the overall distribution is as follows:



· dsleep parameter area: The front is the Deep Sleep function parameter area. · text/data: code segment and data segment of the program. · heap: The heap interval that can be used by the program. The heap interval size can be set according to actual application requirements.

Wi-Fi rx buffer: Wi-Fi rx buffer interval Wi-Fi tx buffer: Wi-Fi tx buffer interval

The size of the Wi-Fi tx/rx buffer can be adjusted according to the size of the tx/rx

data amount. For example, if there is a lot of actual communication tx data and less rx data, you can increase the tx buffer and reduce the rx buffer. The

settings of each buffer interval above are defined in sys_config.h.

```

27:
28: #ifndef SYS_HEAP_SIZE
29: #define SYS_HEAP_SIZE (108*1024)
30: #endif
31:
32: #ifndef WIFI_RX_BUFF_SIZE
33: #define WIFI_RX_BUFF_SIZE (17*1024)
34: #endif
35:
36: #define SYS_HEAP_START (USER_POOL_START)
37: #define WIFI_RX_BUFF_ADDR (SYS_HEAP_START+SYS_HEAP_SIZE)
38: #define SKB_POOL_ADDR (WIFI_RX_BUFF_ADDR+WIFI_RX_BUFF_SIZE)
39: #define SKB_POOL_SIZE (USER_POOL_START+USER_POOL_SIZE-SKB_POOL_ADDR)
40:
41:

```

Heap default size

Wi-Fi rx buffer size

Heap buffer address

Wi-Fi rx buffer location

Wi-Fi tx buffer address

4.8. Basic rules for solution development

Some basic guidelines that need to be followed in solution development include header file references, memory allocation, delays, printing and data formats.

conversion, etc.; in addition, common configurations for program development mainly look at project_config.h and sys_config.h, which contain

System parameters, storage space, peripheral functions and parameters are configured.

Header file reference

When using the SDK for secondary development code, please use the header file defined by the SDK for header file reference. The basic sequence of header file references is

shown in the figure below. typedef.h is a basic type header file. All source code files only need to reference this header file. Do not directly reference the compiler's header file.

list.h, dev.h, and devid.h are the basic header files for device access. The device HAL

api is in the include/hal directory. According to the actual accessed device, add the corresponding header file to reference. osal/string.h, osal/

semaphore.h, osal/xxx are OSAL APIs, and the header files are in include/osal

Table of contents. When you need to use this kind of API, just quote osal/xxx.h.

```

#include "typesdef.h"
#include "list.h"
#include "dev.h"
#include "devid.h"
#include "osal/string.h"
#include "osal/semaphore.h"
#include "osal/mutex.h"
#include "osal/irq.h"
#include "osal/task.h"
#include "osal/sleep.h"
#include "osal/timer.h"
#include "osal/string.h"
#include "hal/gpio.h"
#include "hal/watchdog.h"
#include "lib/common.h"
#include "lib/syscfg.h"
#include "lib/sysheap.h"
#include "lib/skb/skb.h"
#include "lib/skb/skb_list.h"
#include "lib/mac_bus.h"
#include "lib/lmac/lmac.h"
#include "lib/lmac/lmac_host.h"
#include "lib/wnb/libwnb.h"
#include "lib/atcmd/libatcmd.h"
#include "lib/xmodem.h"
#include "lib/mcu/mcu_util.h"

```

Commonly used

APIs for

debugging and printing
 • `os_printf`: Printing API with timestamp
 • `dump_hex`: Print data in hexadecimal format

Memory

allocation
 • `os_malloc`: apply for memory
 • `os_free`: release memory
 • `os_zalloc`: apply for memory and clean it
 • `os_realloc`: Apply for memory, copy the data in the original memory, and release the original memory space.

Sleep function

• `os_sleep`: task sleep API, unit is second
 • `os_sleep_ms`: task sleep API, unit is millisecond

Format

conversion
 • `hex2int`: Convert hexadecimal string to int type data
 • `hex2char`: Convert hexadecimal string to char type data
 • `hex2bin`: Convert hexadecimal string to char array
 • `str2mac`: MAC address string
 • Convert to char array

4.9. Driver API usage instructions

The SDK is designed with a hardware abstraction layer HAL. Applications must use the HAL API to access peripherals. The HAL api will execute the driver-

specific API. Basic steps for an application

to access peripherals: 1. Obtain the device that needs to be accessed: Use the [dev_get](#) API to obtain the peripheral device that needs to be accessed. This API obtains the initialized device through the specified device id. The device id is defined in the include/devid.h file.

2. Use HAL API to access the device. A basic example is as follows:



```
static void watchdog_task(void *args)
{
    struct watchdog_device *wd = (struct watchdog_device *)dev_get(HG_WDTV1_DEVID);
    ASSERT(wd);
    watchdog_device_open(wd, 0);
    watchdog_device_start(wd, 5);
    while (1) {
        watchdog_device_feed(wd);
        os_sleep(2);
    }
}
```

Annotations in the image:

- Red arrow from "get device" to `dev_get(HG_WDTV1_DEVID)`
- Red arrow from "watchdog device" to `wd`
- Red arrow from "FIRE THING" to `wd` in `watchdog_device_start(wd, 5);`

4.10. OSAL API usage instructions

The SDK has designed an OS abstraction layer and provides OSAL APIs for commonly used functions such as semaphore/mutex/timer/irq/msg queue/task. Please use the APIs provided by OSAL when developing code.

4.10.1. Task

Task OSAL API is defined in include/osal/task.h file.

```

1: typedef enum {
2:     OS_TASK_PRIORITY_IDLE           = 0,
3:     OS_TASK_PRIORITY_LOW           = 0x10,
4:     OS_TASK_PRIORITY_BELOW_NORMAL = 0x20,
5:     OS_TASK_PRIORITY_NORMAL        = 0x30,
6:     OS_TASK_PRIORITY_ABOVE_NORMAL = 0x40,
7:     OS_TASK_PRIORITY_HIGH          = 0x50,
8:     OS_TASK_PRIORITY_REALTIME      = 0x60,
9:     OS_TASK_PRIORITY_ISR           = 0xFF,
10: } OS_TASK_PRIORITY;
11:
12: #define OS_TASK_INIT(name, task, func, data, prio, stksize) do { \
13:     os_task_init((const uint8 *)name, task, (os_task_func_t)func, (uint32)data); \
14:     os_task_set_stacksize(task, stksize); \
15:     os_task_set_priority(task, prio); \
16:     os_task_run(task); \
17: }while(0)
18:
19: struct os_task;
20:
21: struct os_task_time {
22:     uint32 id;
23:     const uint8 *name;
24:     uint32 time;
25: };
26:
27: typedef void (*os_task_func_t)(void *arg);
28:
29: int32 os_task_init(const uint8 *name, struct os_task *task, os_task_func_t func, uint32 data);
30: int32 os_task_priority(struct os_task *task);
31: int32 os_task_stacksize(struct os_task *task);
32: int32 os_task_set_priority(struct os_task *task, uint8 priority);
33: int32 os_task_set_stacksize(struct os_task *task, int32 stack_size);
34: int32 os_task_run(struct os_task *task);
35: int32 os_task_stop(struct os_task *task);
36: int32 os_task_del(struct os_task *task);
37: int32 os_task_runtime(struct os_task_time *task_times, int32 count);
38: struct os_task os_task_current(void);

```

• OS_TASK_INIT This

macro definition performs task initialization, stack size/priority setting, and starts running the task. Use this macro

Definitions simplify task creation.

For specific examples, please refer to watchdog task:

```

1: int main(void)
2: {
3:     int8 print_interval = 0;
4:
5:     OS_TASK_INIT("watchdog", &wdog_task, watchdog_task, 0, OS_TASK_PRIORITY_NORMAL, 256);
6:     syscfg_init(&sys_cfgs);
7:     sys_atcmd_init();
8:     sys_wifi_init();
9:     sys_keyled_init();
10: }

```

Annotations in the image:

- task name**: points to "watchdog"
- task object**: points to &wdog_task
- task function**: points to watchdog_task
- task data**: points to 0
- task priority**: points to OS_TASK_PRIORITY_NORMAL
- task**: points to the task object &wdog_task
- stack**: points to 256

• os_task_init: task initialization

int32 os_task_init(const uint8 *name, struct os_task *task, os_task_func_t func, uint32 data)

Parameters: • name: task

name • task: specific task object

• func: task function that needs to be executed •

data: parameters needed to execute the function

Return

value: • RET_OK: initialization

successful • RET_ERR: initialization failed

ÿ os_task_priority: View the priority of the task

int32 os_task_priority(struct os_task *task) Parameters: ÿ

task:

the task object that needs to be viewed

Return

value: ÿ The priority of the task

ÿ os_task_stacksize: View the stack size of the task

int32 os_task_stacksize(struct os_task *task) Parameters: ÿ

task:

the task object to be viewed Return

value: ÿ

The stack size of the task

ÿ os_task_set_priority: Set the priority of the task (the **setting is valid before the task is run**). int32

os_task_set_priority(struct os_task *task, uint8 priority) Parameters: ÿ task: the task object that

needs to be set ÿ priority: the priority of

the task. Priority definition: OS_TASK_PRIORITY Return value: ÿ RET_OK:

Setting

successful ÿ RET_ERR:

Setting failed

ÿ os_task_set_stacksize: Set the stack size of the task (the **setting is valid before the task is run**).

int32 os_task_set_stacksize(struct os_task *task, int32 stack_size) Parameters: ÿ task: the task

object that needs to be set ÿ stack_size:

the stack size of the task Return value: ÿ

RET_OK:

Setting successful ÿ

RET_ERR: Setting failed

ÿ os_task_run: Start running task

int32 os_task_run(struct os_task *task) Parameters: ÿ task: The

task

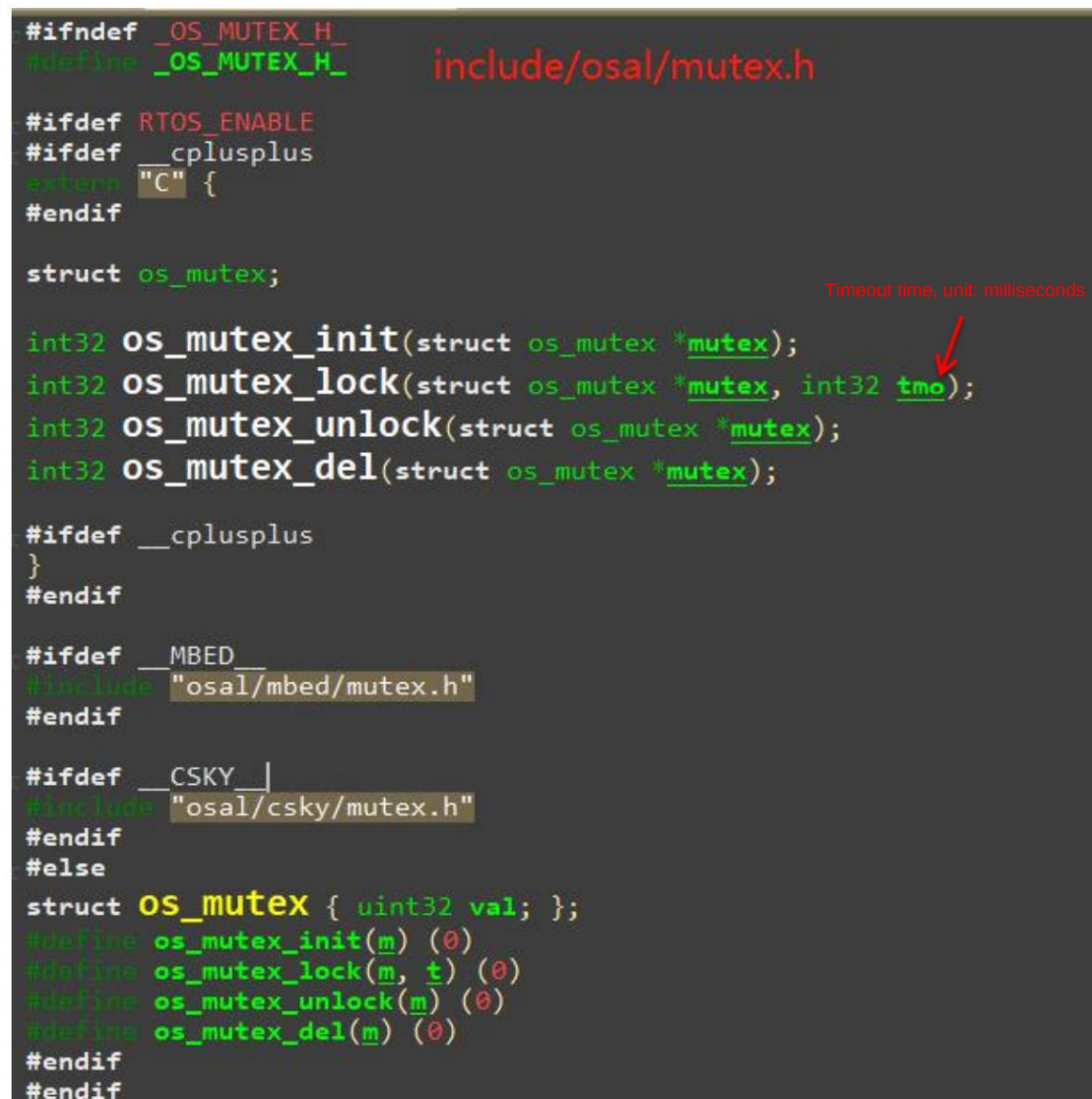
object that needs to be started and run. Return value:

ÿ RET_OK:

The task starts and runs successfully (it does not mean that **the task is executed immediately**). ÿ RET_ERR: The task failed to start and run.

4.10.2. Mutex

The Mutex function OSAL API is defined in include/osal/mutex.h, as shown in the following figure:



```

#ifndef _OS_MUTEX_H_
#define _OS_MUTEX_H_    include/osal/mutex.h

#ifdef RTOS_ENABLE
#ifdef __cplusplus
extern "C" {
#endif

struct os_mutex;

int32 os_mutex_init(struct os_mutex *mutex);
int32 os_mutex_lock(struct os_mutex *mutex, int32 tmo);
int32 os_mutex_unlock(struct os_mutex *mutex);
int32 os_mutex_del(struct os_mutex *mutex);

#ifdef __cplusplus
}
#endif

#ifdef __MBED__
#include "osal/mbed/mutex.h"
#endif

#ifdef __CSKY__
#include "osal/csky/mutex.h"
#endif
#else
struct os_mutex { uint32 val; };
#define os_mutex_init(m) (0)
#define os_mutex_lock(m, t) (0)
#define os_mutex_unlock(m) (0)
#define os_mutex_del(m) (0)
#endif
#endif
  
```

Timeout time, unit: milliseconds

4.10.3. Semaphore

Semaphore function OSAL API is defined in include/osal/semaphore.h

```

#ifndef __OS_SEMAPHORE_H
#define __OS_SEMAPHORE_H           include/osal/semaphore.h

#ifdef __cplusplus
extern "C" {
#endif

struct os_semaphore;

int32 os_sema_init(struct os_semaphore *sem, int32 val);
int32 os_sema_del(struct os_semaphore *sem);
int32 os_sema_down(struct os_semaphore *sem, int32 tmo_ms);
int32 os_sema_up(struct os_semaphore *sem);

#ifdef __cplusplus
}
#endif

#ifdef __MBED__
#include "osal/mbed/semaphore.h"
#endif

#ifdef __CSKY__
#include "osal/csky/semaphore.h"
#endif

#ifndef osWaitForever
#define osWaitForever 0xFFFFFFFFu
#endif
#endif

```

sema initial value, usually 0

Timeout time, unit: milliseconds

wait forever macro definition

4.10.4. Msgqueue

The Message Queue function OSAL API is defined in the include/osal/msgqueue.h file. Message queue stores uint32 type data.

```

#ifndef __OS_MSG_QUEUE_H_
#define __OS_MSG_QUEUE_H_

#ifdef __cplusplus
extern "C" {
#endif

struct os_msgqueue;

int32 os_msgq_init(struct os_msgqueue *msgq, int32 size);
uint32 os_msgq_get(struct os_msgqueue *msgq, int32 tmo_ms);
int32 os_msgq_put(struct os_msgqueue *msgq, uint32 data, int32 tmo_ms);
int32 os_msgq_del(struct os_msgqueue *msgq);
int32 os_msgq_cnt(struct os_msgqueue *msgq);

#ifdef __cplusplus
}
#endif

#ifdef __MBED__
#include "osal/mbed/msgqueue.h"
#endif

#ifdef __CSKY__
#include "osal/csky/msgqueue.h"
#endif

#ifndef OS_MSGQ_DEF
#define OS_MSGQ_DEF(name, size) struct os_msgqueue name;
#endif

#ifndef osWaitForever
#define osWaitForever 0xFFFFFFFF
#endif

#endif

```

include/osal/msgqueue.h

msg queue size

Timeout time in milliseconds

Timeout time in milliseconds

wait forever

• os_msgq_init: Message queue initialization, specify the queue

size. int32 os_msgq_init(struct os_msgqueue *msgq, int32 size); Parameters: •

msgq:

msg queue that needs to be initialized • size:

Specify the size of the msg queue. During initialization, a memory space of size*sizeof(uint32) will be requested from the

heap.

Return value: • RET_OK:

Initialization successful • RET_ERR: Initialization failed

• os_msgq_get: Extract data from msg queue, the timeout can be specified

uint32 os_msgq_get(struct os_msgqueue *msgq, int32 tmo_ms); Parameters: •

msgq:

msg queue to extract data • tmo_ms: get

operation timeout, in milliseconds; or wait forever: osWaitForever return value

• Data provided from msg queue

• `os_msgq_put`: Store data `int32` into msg queue

`os_msgq_put(struct os_msgqueue *msgq, uint32 data, int32 tmo_ms);` Parameters: • `msgq`:

msg

queue that needs to store data • `data`: The

data that needs to be stored •

`tmo_ms`: put operation Timeout, in milliseconds; or wait forever: `osWaitForever` Return

value •

`RET_OK`: Data is successfully stored in

msg queue • Other values: Storage failure.

• `os_msgq_del`: Delete msg queue `int32`

`os_msgq_del(struct os_msgqueue *msgq);`

• `os_msgq_cnt`: View the number of data stored in the current msg

queue. `int32 os_msgq_cnt(struct os_msgqueue *msgq);`

4.10.5. SoftTimer

The OSAL API of Soft Timer is defined in the `osal/timer.h` file.

Soft timer runs based on OS tick, so the time accuracy of the timer is os tick, and the SDK default OS Tick is 1ms.

Notes on using soft timer:

1. Try to shorten the execution time of the timer's callback function as much as possible. All soft timers share 1 execution sequence.

If the execution time of a certain timer is too long, it will affect the execution of other timers.

2. Soft timer has 2 working modes:

• `ONCE`: After the timer is started and triggered, it needs to be started again before it can run.

• `PERIODIC`: The timer only needs to be started once and will be triggered periodically according to the specified timeout period.

```

#ifndef __OS_TIMER_H_
#define __OS_TIMER_H_

#ifdef __cplusplus
extern "C" {
#endif

enum OS_TIMER_MODE {
    OS_TIMER_MODE_ONCE,
    OS_TIMER_MODE_PERIODIC
};

struct os_timer;

typedef void (*os_timer_func_t)(void *arg);

int os_timer_init(struct os_timer *timer, os_timer_func_t func,
                  enum OS_TIMER_MODE mode, void *arg);
int os_timer_start(struct os_timer *timer, unsigned long expires);
int os_timer_stop(struct os_timer *timer);
int os_timer_del(struct os_timer *timer);

#ifdef __cplusplus
}
#endif

#ifdef __MBED__
#include "osal/mbed/timer.h"
#endif

#ifdef __CSKY__
#include "osal/csky/timer.h"
#endif

#endif

```

4.11. Debugging information

The SDK provides basic debugging methods for troubleshooting during the development process.

4.11.1. WiFi protocol stack debugging information output

The underlying debugging information of the WiFi protocol stack is mainly used to analyze wireless transmission problems. Encountered in wireless transmission Problem, please grab the print information for analysis.

```

-----
local:06:07:08:09:12:67
  chip-temperature: 39
  freq:2437, bg_rssi:-76
  gain_table:0
  tx: tx dma:3412, total tx:3410, retry:439, tx lost:5, tx err:0
  rx: rx irq:1263
  rx err: phy err:503, rx frm err:1383,

sta:06:07:08:09:12:22, aid:1, rssi:-37, evm:-24, tx frm type:*3, tx mcs:*5, freq offset:3042
-----

```

The meanings of some of the printed

information are as follows: local:

current MAC address. chip-temperature: current chip

temperature. freq: current working

frequency. bg_rssi: The most recent background noise, in dB. tx: tx

statistics from the last time printed to the present. Among them, tx dma represents the total number of DMA frames; total tx represents the number of frames that have been sent; retry represents the number of air interface retransmissions; tx lost represents the number of frames that failed to be sent; tx

err represents the number of sending errors. rx: rx statistics from the last time printed to the present. rx irq represents the

number of interrupts entering rx receiving frames. rx err: Receive error statistics from rx from the last printing to the present. Due to the large amount of interference

in 2.4G and its complexity, a large number of rx err will occur. sta: represents the information of connected sta. aid is the aid information of sta; rssi represents the signal strength of the latest sta; evm represents the signal quality of the latest received frame (the smaller, the better). Only frames with OFDM modulation mode will be updated; tx frm type and tx mcs are used Yu represents the latest frame format we sent to sta; freq offset represents the latest sta frequency offset information we detected.

Note: The printing time can be set through the AT command (AT+PRINT_PERIOD=__), the unit is ms. When the time is set to 0, printing is turned off.

4.11.2. System Heap usage

Call `sysheap_status()` to print the system Heap usage. The print information will output the current heap remaining status. The heap size can be adjusted based on the printing information. Just keep the heap size sufficient so that the tx/rx buffer can use more memory.

```

[6073] -----
[6076] sysheap free regions: 2
[6079] 01: 0x20056ffc ~ 0x200571a4, size:424      [8]
[6083] 02: 0x200579e4 ~ 0x2005b3d8, size:14836   [13]
[6088] total free size:15260
[6091] -----
[6095] total size:40960
[6098]
LMAC STATUS:

```

System heap usage: heap size: 40K, 15260bytes
remaining, divided into 2 memory slices

4.11.3. CPU usage

Call `cpu_loading_print()` to print the CPU usage. The print information displays the CPU of each Task.

Usage. From this information, you can analyze whether there are any abnormalities in the code operation.

```

[6073] -----
[6075] Task Runtime Statistic, interval:6073ms
[6079] -----
[6084] 1  idle_task      92% (5626)
[6087] 2  timer_task     0% (1)
[6090] 3  MAIN           1% (70)
[6093] 7  lmac tx        2% (159)
[6096] 8  lmac tx status 0% (24)
[6099] 9  lmac rx        0% (48)
[6102] 10 lmac_test      0% (1)
[6105] 13 wrb           2% (125)
[6108] -----
[6112]

```

CPU usage of each Task

4.11.4. XIP instruction fetch efficiency

For solutions that run XIP applications, you need to care about code fetch efficiency, which is mainly affected by SPI FLASH read mode and frequency.

And the impact on Cache hit rate. The SPI FLASH read mode and frequency are configured in the project makecode.ini; Cache

hit rate = 1 - cpmtr/cpfatr. Among them, the number of Cache losses (cpmtr) and hits (cpfatr) can be determined by the function

Use `csi_cache_get_miss_time()` and `csi_cache_get_access_time()` to obtain, when you need to unify a certain segment

When measuring the cache hit rate within a certain period of time, it is recommended to call `csi_cache_reset_profile()` to clear historical data before counting.

When the program code is having difficulty running, you can consider the following optimization directions:

1. Improve SPI FLASH access efficiency (increase line mode, increase frequency, etc.)
2. Optimize code to improve Cache hit rate (enhance code time and space locality)
3. Optimize the execution efficiency of the code itself

5. Other learning resources

[TXW80X Technical Specification](#)

This manual introduces the TXW80X chip, including TXW80X product overview (features, functional block diagram, pin definition and layout, pin functions), functional description (CPU, storage and flash memory, clock, analog and digital peripherals, etc.), electrical Gas parameters (working conditions, DC and AC characteristics, power consumption, reliability, radio frequency performance, etc.) and other information.

[TXW80X Hardware Design Guide](#)

This manual introduces the TXW80X hardware design, including the TXW80X main power supply, RF and high-speed module design and layout guidelines and considerations, schematics, etc.

[TXW80X SDK API Reference Manual](#)

This manual introduces the TXW80X SDK abstraction layer API.

[TXW80X FAQ](#)

This manual introduces the troubleshooting during the development process of the TXW80X solution.

[TXW80X Video Development Guide](#)

This manual introduces the development of TXW80X video solution, including DVP/MJPEG/USB configuration, DVP lens driver, USB Sensor driver, and video and image data streaming.

[TXW80X USB Development Guide](#)

This manual introduces the TXW80X USB solution development, including USB Host and USB device initialization and work Operation process, etc.

[TXW80X Bluetooth distribution network development guide](#)

This manual introduces the development of TXW80X Bluetooth distribution network.

[TXW80X Audio Development Guide](#)

This manual introduces the development of the TXW80X audio solution, including the introduction of the audio daughter board and the use of the PDM/IIS audio interface. Use and audio data stream, etc.

[TXW80X IOT Development Guide](#)

This manual introduces the TXW80X IOT solution development.

[TXW80X Low Power Development Guide]

This manual introduces the development of TXW80X low power consumption solution.

TXW80X Certification Test Guide

This manual introduces the TXW80X solution certification test.

[TXW80X Antenna Matching and Testing Guide]

This manual introduces the TXW80X solution certification test.

[TXW80X mass production and programming guide]

This manual explains in detail the configuration, tools and processes related to mass production programming of TXW80X, including online and offline programming.

Burn.